PokemonPedia improvement plan

Developer guide | 11 October 2026 | Budget requirement: free

Make the catalogue load reliably, correct evolution relationships, and make every essential action work with touch and keyboard before adding more features. The existing React and Express application provides a useful foundation, but its data lifecycle and interaction model need work before a dependable public release.

What exists today

The app has a catalogue with name search, all 18 type filters, legendary mythical and baby filters, three card layouts, a two Pokemon comparison page, and an evolution page. TanStack Query manages catalogue requests, Radix components provide several controls, and the shared schema centralizes type names and colors.

Verified local results

npm run check fails with TS2322 at server/vite.ts:39 because allowedHosts is inferred as boolean instead of true or string[]. npm run build succeeds. The emitted main JavaScript is 517.14 kB, or 161.20 kB gzip, and Vite warns that it exceeds 500 kB. CSS is 65.19 kB, or 11.29 kB gzip. The build also reports outdated Browserslist data.

Browser evidence

The catalogue was inspected at the normal desktop viewport and an emulated 390 by 844 viewport. Both showed a loading catalogue with a zero count while initialization continued. The accessibility tree exposed type chips as text rather than keyboard controls and the search icon as an unnamed button. A click on the Legendary switch changed its state successfully; wrapper click handling is a cleanup risk, not a confirmed broken toggle.

How to use this plan

P0 means a release blocker, P1 means a major reliability or usability fix, P2 means the next improvement pass, and P3 means optional expansion. Complete the required backlog before optional features. Effort estimates are planning ranges for one developer familiar with React and TypeScript, not delivery commitments.

Preserve these strengths

Keep typed components, shared type definitions, the useful compare and evolution routes, debounced catalogue search, image alt text, existing grid breakpoints, skeleton loading, and the fan attribution footer. Improve these foundations instead of replacing the entire stack.

Prioritized work backlog

Implement in dependency order. Tasks marked required belong in the first dependable release. The detailed sections explain the changes and the checks needed to close each task.

P0 required

R01 Fix the TypeScript failure. R02 Remove the full sequential import from the first user request. R03 Make catalogue initialization complete, persistent, and safe under simultaneous requests. R04 Preserve real evolution branches and species identity.

P1 required

R05 Add distinct loading failure empty and partial data states. R06 Protect evolution selection from stale requests. R07 Replace hover only details with an accessible detail action. R08 Add keyboard semantics labels and selection states to controls. R09 Correct badge contrast and permit zoom. R10 Repair lazy audio loading and make playback explicit. R11 Validate API inputs and bound upstream requests. R12 Handle server errors without rethrowing after the response.

P2 next pass

R13 Replace the fixed catalogue boundary with a versioned index. R14 Correct data units and species form modeling. R15 Add name and ID search with resettable URL state. R16 Improve comparison selection and useful statistics. R17 Add pagination or bounded rendering. R18 Split routes and reduce unused dependencies. R19 Consolidate layout tokens fonts and common components. R20 Improve mobile density and evolution layout. R21 Add image fallback and complete lazy loading. R22 Document setup and add meaningful automated checks. R23 Prepare free static deployment and metadata. R24 Add lightweight diagnostics and freshness controls.

P3 optional

F01 Favorites and caught tracking. F02 Team builder and defensive type coverage. F03 Shiny and form selection. F04 Offline catalogue. F05 Quiz and random discovery. F06 Localization and game specific data.

Suggested implementation order

First: R01 to R03 and R05. Second: R04 R06 R13 and R14. Third: R07 to R12. Fourth: R15 to R24. Finally: choose optional features based on actual use. A feature that depends on new data fields must follow the data model and cache work.

First useful milestone

A new user can open the catalogue, search and filter, open a detail view, compare two entries, and inspect a correct evolution family without a long cold import or a hidden failure. This milestone is more valuable than adding login or decorative animation.

Catalogue data and cache reliability

R02 P0 remove request time importing

Evidence: server/routes.ts:11 and :153 loop over 1 to 1010; each iteration awaits Pokemon then species requests. A complete successful run makes 2020 upstream calls sequentially before the initial response. Work: generate a cached catalogue before serving visitors, or serve a small index immediately and fetch details on demand. Acceptance: a fresh visit does not wait for a full catalogue import and does not initiate thousands of upstream calls.

R03 P0 make initialization atomic and persistent

Evidence: server/storage.ts uses Maps; routes.ts starts initialization when storage is empty and writes entries one at a time. A second request can trigger duplicate initialization or receive a partially filled Map; any nonempty result then bypasses initialization. Work: build a temporary snapshot, track readiness and failed IDs, publish atomically, and persist it. If retaining Express, share one initialization promise and retry only missing records. Acceptance: simultaneous cold requests produce one import, restarts retain data, and failed records are retried without silently presenting an incomplete catalogue as complete.

R13 P2 replace the fixed boundary

Evidence: routes.ts hard codes 1010 and describes that range as all Pokemon up to generation 9. Work: discover species through a cached versioned index; record supported generations, record count, refresh date, and excluded forms. Do not confuse Pokemon resource IDs with species IDs. Acceptance: the UI describes its actual dataset, and adding entries does not require changing a loop boundary.

R14 P2 preserve data meaning

Evidence: shared/schema.ts declares integer height and weight, while routes.ts converts them to fractional meters and kilograms. MemStorage hides the database mismatch. Work: store original integer units and convert for display, or use decimal database columns. Add speciesId, canonical slug, display name, generation, form identity, ability hidden status, nullable media, and optional base stats. Validate normalized JSON with Zod rather than relying on casts. Acceptance: fractional measurements round trip correctly and missing artwork does not break a card.

Cache design for a free project

Use a small versioned catalogue index plus individual detail JSON files and evolution family JSON files. Keep a manifest with schemaVersion, generatedAt, expectedCount, missingIds, and content version. Refresh through an explicit command and publish only a valid snapshot. Keep the last known good snapshot if refresh fails. Cache upstream resources locally as requested by the PokéAPI fair use policy [1].

Correct evolution behavior

R04 P0 render the actual family graph

Evidence: client/src/pages/Evolution.tsx:230 recursively flattens all children into names, :153 sorts matches by national ID, and :408 draws an arrow between consecutive entries. That can imply evolution between sibling branches, and ID order is not evolution order. Work: preserve parent child edges from the response, render sibling branches separately, and connect only actual edges. Acceptance: Eevee has independent branches; Pichu precedes Pikachu despite its higher ID; selecting any family member shows the same correct family.

Use species identity rather than fuzzy names

Evidence: Evolution.tsx:80 compares a lowercased Pokemon name with capitalizeFirstLetter(speciesName), so the exact comparison cannot match ordinary names. Later includes and prefix matching can choose an unrelated or wrong form. Work: extract stable species IDs, map them to default Pokemon varieties, and keep display names separate from identity. Remove speculative name matching and hard coded starter family fallbacks. Acceptance: hyphenated names and regional varieties resolve deterministically without false relationships.

Show the conditions on each edge

The page currently omits evolution_details from its local interface. Work: retain edge conditions and format multiple alternatives clearly, including level item friendship and other supported conditions. Label the game or version context when conditions differ. Acceptance: conditions appear next to the relevant parent child edge, null fields are omitted, and multiple alternatives are not merged into one invented rule. The upstream chain includes children and edge details [1].

R06 P1 prevent stale selections

Evidence: Evolution.tsx uses an asynchronous useEffect path without cancellation or a request identity check. A slower earlier selection can overwrite a newer selection; a failed request can leave the previous family visible. Work: query by chain ID through TanStack Query, propagate AbortSignal, clear or explicitly mark old data, and show retryable errors. Choose either the existing server proxy or the static data source consistently. Acceptance: rapid selection changes always finish on the latest Pokemon and request failure never appears as a legitimate no evolution result.

Evolution search and mobile behavior

Trim input, reject blank search, support Enter, show selectable matches, and explain no match. Replace the fixed width horizontal sequence with a stacked branch layout on narrow screens and a readable tree on wide screens. Keep any horizontal scrolling contained and keyboard accessible. Single stage families should say that there are no further evolutions rather than that data is unavailable.

Catalogue discovery details and comparison

R05 P1 distinguish request states

Evidence: Home.tsx consumes data and isLoading but not isError; Compare and Evolution also ignore catalogue failure. PokemonGrid treats an empty array as no matching Pokemon. Work: define loading, success, no matches, request failure, and incomplete data states. Include Retry for failures and Clear filters for no matches. Disable selectors during loading. Acceptance: a rejected request never produces a misleading zero result catalogue or a silent empty selector.

R07 P1 create an accessible detail destination

Evidence: PokemonCard.tsx uses onMouseEnter and onMouseLeave on a div to reveal description height weight and ability. Touch and keyboard users lack an equivalent action. Work: add a real link to /pokemon/:id or an explicit Details button with an accessible dialog. Keep hover as a small enhancement only. Acceptance: the same essential information is available by tap and keyboard; a detail URL opens correctly after refresh.

R15 P2 make search and filters predictable

Evidence: Home.tsx searches names only and sorts only by ID. SearchBar has a button with no onClick. Work: support name slug and numeric ID, trim and normalize input, explain special filter OR behavior, add Clear all and active filter indicators, and offer ID name height and weight sort. Save query type sort and layout in URL parameters so Back and share preserve context. Acceptance: 25 and Pikachu resolve the same entry; whitespace does not prevent matching; reset clears both search text and filter state.

R16 P2 make comparison useful

Evidence: Compare.tsx renders two large unsearchable lists and repeats the same panel markup. Work: build a searchable combobox, allow swap clear and preselection from cards, and extract a reusable comparison panel. Show aligned base stats and totals when added to the model; highlight differences without suggesting that a higher stat proves a battle outcome. Save selections in the URL. Acceptance: users can choose entries by name or ID, use the selector with a keyboard, and reopen a shared comparison.

Details worth adding after stability

Display full abilities with hidden status, six base stats, normalized measurements, family navigation, generation, forms, and explicit cry playback. Add type strengths and weaknesses only with correctly combined dual type multipliers and stated game context. Fetch moves and encounters separately so opening a simple detail view remains fast. Acceptance: missing optional information is labeled unavailable and does not prevent the rest of the page from working.

Accessibility and interaction requirements

R08 P1 make controls semantic

Evidence: FilterBar.tsx renders clickable Badge components; ui/badge.tsx:32 returns a div. Work: use native buttons with aria-pressed, visible focus, and a labelled filter group. ViewToggle already has accessible names; add selected state with aria-pressed or a radio group. Acceptance: Tab reaches every filter, Enter and Space activate it, and a screen reader announces the selected state. Relevant WCAG areas: Keyboard and Name Role Value [2].

Label search selectors and icon links

Evidence: SearchBar.tsx has no associated input label and an unnamed icon button. Compare and Evolution labels lack explicit linkage to their input or trigger. Footer.tsx icon links have no accessible text. Work: add visible or visually hidden labels; wire htmlFor to IDs; make search a form with a working labelled submit action or remove the decorative button; name external links. Acceptance: accessibility inspection gives every interactive element a meaningful name and form labels are announced.

R09 P1 correct contrast and zoom

Evidence: white small type labels appear on bright backgrounds; white on electric #F8D030 computes to about 1.49 to 1 before any shadow treatment. The text-shadow utility is referenced but has no definition in the inspected styles. client/index.html sets maximum-scale=1. Work: choose dark text on light type colors or darker badge surfaces, verify each pairing, and remove the zoom restriction. Acceptance: normal text meets a 4.5 to 1 target; text resizing and zoom preserve content and controls. Relevant WCAG areas: Contrast Minimum Resize Text and Reflow [2].

Touch targets motion and status

Evidence: compact view buttons and type chips are visibly small; hover transitions have no explicit reduced motion treatment in the inspected styles. Work: use a comfortable 44 px touch target design goal, while checking the applicable WCAG target size rules separately; add focus-visible styling and a deliberate reduced motion variant. Announce result counts and loading errors politely without reading the whole grid. Acceptance: controls can be used at mobile size, zoom, and reduced motion without accidental actions.

Preserve working primitives

Radix switches already have connected labels and the tested Legendary switch changed state. Simplify clickable wrapper handling to avoid competing handlers, then test switch and label clicks individually. Do not report this as a confirmed double toggle. Verify dialog focus return, combobox keyboard operation, and native links rather than assuming library use makes the whole page accessible.

Visual design and responsive usability

R19 P2 create a small consistent design system

Evidence: Home Compare Evolution PokemonCard FilterBar and Footer repeat hard coded dark colors and duplicated header markup, while Tailwind already defines semantic tokens. Work: create AppShell Header PageHeading TypeBadge PokemonSummary and reusable selector components. Define background surface text muted text borders focus rings spacing radii and type badge text colors centrally. Acceptance: changing one token updates every route and reusable components stay consistent.

Repair typography and identity

Evidence: pages request font-roboto and font-[Poppins], but the inspected Tailwind config defines no Roboto family and the app does not load those fonts. The desktop screenshot shows a serif heading fallback. Work: choose a reliable system font stack or self host a free font, set heading and body styles intentionally, and standardize PokemonPedia versus PokeDex naming. Acceptance: fonts load consistently, headings have a clear hierarchy, and branding metadata matches the visible app.

R20 P2 improve mobile density

Evidence: at 390 by 844 the stacked header and expanded filters occupy much of the first viewport; the skeleton grid begins far below the header. Existing breakpoints do stack content successfully. Work: keep search easy to reach, use a compact filter drawer or expandable section, show active filters and a result count, and test sticky header height. Do not hide applied filters or make resetting them difficult. Acceptance: narrow screens show useful results promptly and filter controls remain discoverable.

Keep details stable and readable

Evidence: grid details animate max-height up to max-h-48, which can clip long descriptions, and revealing information changes card height. Work: use the new details destination for full content, keep catalogue cards stable, clamp previews deliberately, and display a clear details affordance. Provide a compact list view with aligned name ID types and action columns. Acceptance: long descriptions do not clip essential content and hovering one card does not unexpectedly disrupt reading the catalogue.

Design verification targets

Test 320 390 768 1024 and 1440 px widths, long names, dual types, missing artwork, empty results, errors, large text, and focus indicators. Inspect compare columns on tablets and evolution branches on phones. The current viewport observations cover the catalogue loading state; exercise loaded data and actual touch gestures as implementation acceptance work.

Design audit interpretation

The mechanical Impeccable detector returned an empty findings array for the inspected targets. It did not detect the verified keyboard semantics font loading or contrast problems. Keep manual code and browser evidence alongside mechanical checks; an empty detector result is not an accessibility or design certification.

Performance images and audio

R17 P2 bound catalogue rendering

Evidence: Home.tsx renders the full filtered list and PokemonGrid maps every entry to a card. Every card also creates a Howl instance and timer. Work: render a bounded page of roughly 30 to 60 cards, then offer Load more or accessible pagination; use virtualization only if a measured need justifies it. Keep filtering across the full index rather than only loaded pages. Acceptance: initial card mounts are bounded, all entries remain discoverable, and changing filters resets pagination correctly.

R18 P2 split and trim the bundle

Evidence: App.tsx imports every route eagerly; the build emitted a 517.14 kB main JavaScript chunk. package.json contains several backend auth database and UI libraries that may be template leftovers. Work: lazy load Compare Evolution and details, inspect a bundle report, and remove dependencies only after confirming they are unused. Keep schema runtime validation separate from database schema imports where that reduces client weight. Acceptance: the initial chunk is smaller and navigation loading feedback works; build and type checks still pass.

R10 P1 repair audio lifecycle

Evidence: useAudio.ts starts a 3 second timeout even with preload=false. An untouched card can unload its sound before a user tries to play it; isLoaded is captured by the effect and not reset consistently across source changes. PokemonCard plays on hover, and pointer exit does not stop playback. Work: create and load audio only after an explicit Play cry action, start timeout when loading begins, handle play and load errors, stop previous playback through one controller, and reset state on source changes. Acceptance: waiting more than 3 seconds before first playback works and only one cry plays at a time.

Use reliable media identity

Evidence: routes.ts truncates many hyphenated names when constructing sound URLs; PokemonCard guesses another URL for early species. Work: prefer a supported media URL from normalized upstream data where available, with a real fallback and matching audio format; PokéAPI exposes cry fields [1]. Replace the hard coded mp3 format when using ogg. Do not assume a guessed name maps to the correct cry. Acceptance: special names and forms are tested and unavailable sound is handled quietly but visibly.

R21 P2 improve images and render costs

Grid artwork is lazy loaded, but list artwork is not. Add lazy loading there, explicit reserved dimensions, decoding hints, and an onError placeholder that cannot loop. Cache normalized search text, index by ID, and memoize derived sorting only after measuring. Acceptance: list view does not eagerly fetch the entire catalogue artwork and broken images preserve layout.

Performance gates

Record cold and warm first result time, transferred JavaScript, image requests, mounted cards, and selection responsiveness on a throttled phone profile. Set targets from this baseline. The current build sizes are measured; no Lighthouse score or first contentful paint value is claimed.

Code structure API safety and maintenance

R01 P0 restore a clean type check

Evidence: npm run check reports TS2322 at server/vite.ts:39. Work: type serverOptions using the supported Vite ServerOptions type or preserve the literal type while choosing a restricted host configuration. Do not silence the error with any. Acceptance: npm run check exits zero and development startup still works. Keep type checking separate from a successful transpiling build.

R11 P1 validate and bound requests

Evidence: routes.ts uses parseInt without bounds for limit and offset; the evolution proxy accepts an arbitrary path ID; axios requests have no explicit timeout. Work: validate positive chain IDs and nonnegative pagination, cap limit, return a consistent 400 error for malformed input, set timeouts, and add bounded retry backoff for transient upstream errors. Cache evolution responses. Acceptance: negative malformed and huge inputs are rejected and upstream failure finishes within a defined time budget.

R12 P1 handle failures safely

Evidence: server/index.ts sends an error response and then throws the error, risking an uncaught exception. Work: log the error once, respond once, preserve process availability, handle startup rejection deliberately, and distinguish upstream unavailable from missing data. Add readiness and health endpoints if retaining the server. Acceptance: a controlled failing endpoint does not crash the server and subsequent valid requests still succeed.

Clean template leftovers deliberately

Evidence: shared/schema.ts includes users with a password text field and storage provides user methods, but the app has no visible account flow. Work: remove unused auth and database scaffolding from the free static path. If accounts are added later, design password hashing sessions and recovery as a separate feature; do not build on plaintext password storage. Declare directly imported server packages such as nanoid explicitly rather than depending on a transitive installation. Acceptance: a clean install resolves all direct imports without unnecessary service setup.

R24 P2 keep diagnostics useful

Evidence: server/index.ts stringifies the full API response before truncating logs; queryClient.ts sets staleTime Infinity and retry false; evolution logs raw responses. Work: log request duration status count and cache outcome rather than full payloads, adopt a deliberate freshness policy, expose snapshot version, and remove noisy debugging. Restrict development hosts and production security headers if keeping Express. Acceptance: logs identify failures without serializing every catalogue entry and data refresh is predictable.

R22 P2 make the project maintainable

Add a README covering setup commands Node version data refresh architecture and deployment; an environment example if a server remains; scripts for lint format tests and CI; and a reproducible lockfile workflow. The start script uses shell specific NODE_ENV assignment; use cross-env for Windows compatibility, as dev already does. Make server port configurable instead of fixed at 5000. Audit dependencies and review upgrades individually. No vulnerability result is asserted by this review.

Optional feature roadmap

These additions use free libraries and data, but still cost development time. Build them after the required reliability and usability work. Avoid introducing accounts paid APIs subscriptions or a database just to add personal features.

F01 favorites and caught tracking

Value: gives users a reason to return. Work: add favorite and caught controls, localStorage with a versioned schema, filtered views, and JSON export import. Effort: about 1 to 2 days after detail actions exist. Acceptance: choices survive reload, missing storage permissions do not crash the app, import validates data, and users understand that local choices do not synchronize across devices.

F02 team builder

Value: makes comparison useful beyond two entries. Work: allow up to six selections, summarize defensive weaknesses resistances and immunities from combined types, and export a shareable URL or JSON. Effort: about 3 to 5 days after stats and type relations exist. Acceptance: duplicate and empty slots work predictably, dual type multipliers are correct, and the app does not promise battle outcomes from base stats alone.

F03 shiny artwork and form browsing

Value: improves visual discovery. Work: retain species form relationships, distinguish default regional and other forms, add a shiny switch and image fallback. Effort: about 2 to 4 days after R14. Acceptance: switching forms updates matching data and media; missing shiny artwork does not break details; species counts remain separate from form counts.

F04 offline catalogue

Value: makes the reference usable without a connection. Work: add a manifest and service worker, cache the shell and a controlled dataset, show offline status, and implement a versioned update flow. Effort: about 2 to 4 days after static deployment. Acceptance: previously cached entries work offline, uncached data is explained, and a new deployment does not leave incompatible stale files.

F05 quiz and random discovery

Value: adds a lightweight fun mode. Work: provide random entry discovery or image and type quizzes using the cached index, accessible controls, and local scores. Effort: about 1 to 3 days. Acceptance: answer options are valid distinct and keyboard accessible; loading and missing artwork never reveal a misleading question.

F06 localization and game specific views

Value: broadens usefulness for more players. Work: add translated display names and descriptions, language fallback, locale aware formatting, and game specific moves encounters and evolution conditions. Effort: about 4 to 8 days for a focused initial scope. Acceptance: selected language and game are visible and unsupported combinations fall back explicitly.

Features to defer

Defer social feeds multiplayer accounts battle simulation and an admin dashboard until there is demonstrated demand. They require substantially more state moderation testing or hosting operations than a free reference app needs.

Free architecture and deployment plan

Recommended architecture

Use Vite React and TypeScript as a static client. A local data generator fetches and validates the index detail records and evolution trees, writes versioned JSON under client/public/data, and keeps a last known good cache. The browser requests these static files, uses TanStack Query for caching, and stores favorites locally. This removes the request time import and the need for a persistent paid server or database.

What changes in this repository

Create scripts/sync-pokemon-data.ts and a shared API data model with Zod validation. Add npm scripts for data refresh and client build. Replace /api/pokemon with a static data adapter; replace the client direct evolution request and Express proxy with cached family files. Separate Express code from the static build path, then remove it only after parity is verified. Preserve the server option for local experiments if useful.

Free hosting option

GitHub Pages hosts static files and is available for public repositories on GitHub Free [3]. It cannot run this Express server. Deploy only the client and static data after conversion. Use the included github.io address rather than buying a domain. Confirm that the repository and generated data are suitable for public access before publishing.

Routing and base paths

A repository site normally runs below /repository-name/. Configure Vite base and the router base consistently, use relative data and asset paths, and choose a tested hash route or static fallback approach. Acceptance: home comparison evolution and detail links work when opened directly and after refresh; a deployed asset path never accidentally points to localhost or /api.

Cost and resource controls

All core work can use free tools. Keep refresh manual at first; cache upstream responses and retry only failures. Do not fetch every record on every visit or deployment. Verify current platform quotas before enabling automatic refresh. Public media remains dependent on upstream availability; offline caching should be bounded and intentional. Free hosting does not mean unlimited bandwidth or guaranteed availability.

R23 P2 finish public presentation

Add an accurate title description favicon social preview and consistent application name in client/index.html. Remove the unconditional Replit banner script outside its intended development environment. Replace generic GitHub and Twitter footer destinations with actual project links or remove them; describe PokeAPI as a data source without implying it is an official Pokemon product. Keep the fan attribution. Add a deployment smoke test and a rollback to the previous validated snapshot.

Definition of a free release

The deployed reference works without a paid service custom domain paid API or paid database. Optional automated builds and future private hosting must be reviewed against their then current limits. No paid account or infrastructure change is required by this plan.

Execution checklist and acceptance tests

Phase 1 dependable data and builds

Allow roughly 3 to 5 focused developer days for R01 to R03 R05 R13 and R14. Deliver a validated persisted snapshot a clean type check and distinguishable request states. Test empty cache simultaneous requests upstream timeout missing IDs restart fractional units and malformed pagination. If choosing the static architecture, implement the generator here rather than first investing in a database.

Phase 2 correct usable interactions

Allow roughly 3 to 5 days for R04 R06 to R12 R15 and R20. Deliver correct evolution families tap and keyboard details explicit audio and labelled controls. Test Eevee Pichu a single stage family hyphenated names rapid selection blank search no match every filter label playback after waiting and a controlled error. Test zoom keyboard focus and badge contrast.

Phase 3 polish performance and deployment

Allow roughly 3 to 5 days for R16 to R19 and R21 to R24. Deliver searchable comparison bounded cards route splitting common components documentation and static deployment. Test a clean install build route refresh repository base path missing images favorites storage if enabled and mobile layouts. These ranges overlap and require adjustment after implementation begins.

Meaningful automated coverage

Add Vitest tests for data normalization ID search combined filter rules graph preservation and type multipliers when implemented. Use local fixtures to test failure and race handling without relying on live upstream data. Add a small Playwright suite for catalogue search keyboard detail access comparison selection evolution branches and deployed route refresh. Run type check build and focused tests in CI. Avoid tests that only repeat component implementation.

Release acceptance checklist

Every required backlog item has its acceptance checks recorded. Cold visits avoid full imports. Failed refresh preserves good data. Evolution edges are correct. Search and filters work by keyboard and touch. Errors are retryable. Type check and build pass. Initial rendering is bounded. Cry playback is user initiated. Direct deployed URLs work. README explains refresh and deployment. No paid service is needed.

Official references

[1] PokeAPI v2 documentation and fair use policy: https://pokeapi.co/docs/v2 . Supports cache policy, pagination, species resources, evolution chain shape and media fields. [2] W3C WCAG 2.2 quick reference: https://www.w3.org/WAI/WCAG22/quickref/ . Supports accessibility acceptance criteria. [3] GitHub Pages documentation: https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages . Supports the free public repository static hosting option. References checked on 11 October 2026.

Where to start

Fix R01 and implement the cached data generator first. Then replace evolution flattening with species keyed graph data and add accessible detail actions. Keep optional features behind this foundation. For a later design pass, use Impeccable harden optimize adapt clarify and polish in that order, then repeat the audit.